

LIGHTS OUT PUZZLE

BBC micro:bit v2 • ARM Cortex-M4 Assembly

Computer Architecture Project

Course	Computer Architecture - EGC 121	
Platform	BBC micro:bit v2 (nRF52833, ARM Cortex-M4)	
Language	ARM Assembly (Thumb-2 / unified syntax)	
Members	Abhinav P	IC2025021
	Y B Siddharth	BC2025116

1. Project Overview

Lights Out is a classic single-player puzzle game implemented entirely in ARM Assembly on the BBC micro:bit v2. The 5×5 LED matrix serves as the game board. Each LED represents a cell that is either ON (lit) or OFF. When the player toggles a cell, that cell and all four of its orthogonal neighbours (up, down, left, right) flip their state. The objective is to turn every LED off.

The project was chosen because it exercises several core computer architecture concepts simultaneously — memory-mapped I/O, direct bit manipulation, hardware timers, and LED multiplexing — all without relying on any runtime library or operating system. Every peripheral is accessed by reading and writing to specific memory addresses as defined in the nRF52833 datasheet.

Controls

Button A (short press)	Move cursor forward — left to right, top to bottom, wrapping at position 24 back to 0
Button A (long press >400ms)	Abandon the current puzzle and generate a new one
Button B (short press)	Toggle the cell at the cursor and its four neighbours
Win condition	All 25 LEDs off — triggers a clockwise spiral animation followed by three full-board flashes

2. Architecture Concepts Demonstrated

2.1 Memory-Mapped I/O

The nRF52833 microcontroller exposes every peripheral — GPIO pins, timers, clock — as a set of registers at fixed memory addresses. There is no special I/O instruction; peripherals are controlled by ordinary load (LDR) and store (STR) instructions to specific addresses. This is the defining feature of memory-mapped I/O.

Key base addresses used in this project:

GPIO Port 0 (P0)	0x50000000
GPIO Port 1 (P1)	0x50000300
TIMER0	0x40008000
Clock controller	0x40000000

To set a pin HIGH, the program writes the pin's bitmask to the OUTSET register:

```
ldr r0, =0x50000000    @ GPIO0 base address
ldr r1, =(1 << 21)      @ ROW1 = P0.21
str r1, [r0, #0x508]    @ write to OUTSET → drives pin HIGH
```

The OUTSET/OUTCLR register pair is a hardware optimisation: writing to OUTSET sets only the specified bits without disturbing others, and OUTCLR clears them. This avoids the read-modify-write cycle that would otherwise be needed.

2.2 Bit Manipulation

The game board is stored as five bytes in RAM — one byte per row — where each bit represents whether the LED in that column is ON (1) or OFF (0). All game logic operates directly on these bits using ARM bit instructions.

Toggling a single LED at (row, col) uses an EOR with a shifted mask:

```
ldrb r3, [r2, r0]      @ load grid[row]
mov r12, #1
lsl r12, r12, r1        @ mask = 1 << col
eor r3, r3, r12         @ flip that bit
and r3, r3, #0x1F       @ keep only 5 bits
strb r3, [r2, r0]       @ store grid[row]
```

The win check ORs all five grid bytes together. If the result is zero, every bit across every row is 0, meaning every LED is off and the puzzle is solved. This is an efficient branchless test: a single comparison against zero after five OR operations.

2.3 LED Multiplexing

The 5×5 LED matrix uses only 10 GPIO pins through a multiplexed grid of 5 rows and 5 columns. A physical LED lights up only when its row pin is HIGH and its column pin is LOW simultaneously. Because multiple LEDs share the same column wire, it is impossible to light an arbitrary subset of LEDs on different rows at the same time.

The solution is time-division multiplexing: the program activates one row at a time, sets the correct column pins for that row, waits for one MUX_SLICE (80,000 timer ticks ≈ 5 ms), then moves to the next row. At 16 MHz the full five-row cycle completes in approximately 25 ms, well above the 60 Hz threshold for flicker-free perception.

2.4 Hardware Timer and Precise Delays

TIMER0 is configured as a 32-bit free-running (continuously running after initialization) counter at 16 MHz (prescaler = 0, BITMODE = 3). All time-critical delays — multiplexing slice, button

debounce, long-press detection — are measured in hardware ticks rather than software loop counts. The `delay_ticks` routine captures the current counter value, then spins until the elapsed count reaches the requested number of ticks. This makes delays independent of instruction mix and pipeline behaviour.

2.5 Pseudo-Random Number Generation (LFSR)

Puzzle generation uses a 16-bit Galois Linear Feedback Shift Register with feedback polynomial `0xB400` (taps at bits 15, 13, 12, and 10). This produces a maximal-length sequence of 65,535 states before repeating, provided the initial seed is non-zero. Each call to `lfsr_next` shifts the register right by one bit; if the outgoing bit is 1, the register is XORed with the polynomial.

Puzzle solvability is guaranteed by construction: the grid starts fully OFF and ten random toggle operations are applied. Because every toggle is its own inverse, the resulting puzzle can always be solved by repeating those same ten moves in any order.

3. Implementation

3.1 Memory Layout

The program is organised into four linker sections:

Section	Location	Contents
<code>.text</code>	Flash (ROM)	ISR vector table, all executable code
<code>.rodata</code>	Flash (ROM)	Row/column lookup tables, spiral animation sequence
<code>.data</code>	RAM	Game grid (5 bytes), cursor position, blink counters, LFSR state
<code>.stack</code>	RAM	2 KB call stack, growing downward from <code>_stack_top</code>

3.2 Pin Mapping

All GPIO pin assignments follow the official micro:bit v2 schematic:

Signal	nRF52833 Pin	Bitmask	Direction
ROW1–ROW5	P0.21/22/15/24/19	Bit masks per row	Output (HIGH = active)
COL1–COL3,5	P0.28/11/31/30	Bit masks on GPIO0	Output (LOW = active)

COL4	P1.05	Bit 5 on GPIO1	Output (LOW = active)
Button A	P0.14	Bit 14 on GPIO0	Input with pull-up
Button B	P0.23	Bit 23 on GPIO0	Input with pull-up

COL4 is on GPIO Port 1 (P1.05) rather than Port 0. The code handles this transparently by maintaining two parallel lookup tables — `col_gpio0` and `col_gpio1` — indexed by column number, with a zero entry in whichever table does not own that column.

3.3 Startup Sequence

On reset, the processor executes the following initialisation sequence before entering the game loop:

- `clock_init` — starts the 16 MHz high-frequency crystal oscillator and polls the `HFCLKSTARTED` event register until the clock is stable.
- `gpio_init` — configures all row and column pins as outputs via `DIRSET`, configures both button pins as inputs with internal pull-ups via `PIN_CNF`, and clears the display.
- `timer0_init` — sets `TIMER0` to 32-bit mode, prescaler 0 (16 MHz), then issues `CLEAR` and `START` tasks.
- `new_puzzle` — clears the grid, generates a random solvable puzzle using the LFSR, resets the cursor to position 0.

3.4 Main Game Loop

The main loop runs indefinitely and performs the following steps on every iteration:

- `display_frame` — multiplexes all five rows, applying cursor blink to the active row.
- `update_blink` — increments a frame counter and toggles `blink_state` every 40 frames.
- `check_win` — ORs all five grid bytes; if zero, runs `win_animation` then `new_puzzle`.
- `read_btn_a` — returns 0 (not pressed), 1 (short press → move cursor), or 2 (long press → new puzzle).
- `read_btn_b` — returns 1 on short press, calls `toggle_cell` at the cursor position.

4. Challenges and Solutions

Cross-bank column wiring

COL4 is routed to GPIO Port 1 rather than Port 0, which is unusual. A single display function cannot assume all columns share a base address. The solution was two parallel lookup tables — `col_gpio0` and `col_gpio1` — where only one entry per column is non-zero. The `light_row_mask` function checks both tables for every column bit.

Button debounce without a timer interrupt

Mechanical buttons produce multiple transitions within a few milliseconds of a press. The `read_btn_a` and `read_btn_b` routines handle this by ignoring the initial edge. After detecting an initial button press, the routine waits 640,000 `TIMER0` ticks (~40 ms) using the hardware timer `delay_ticks` routine, then samples the button again to confirm a stable press. Only if the button is still held after debounce does the code proceed to long-press detection.

Guaranteed puzzle solvability

A naive random fill of the grid can produce unsolvable configurations — the Lights Out puzzle has exactly $2^{12} = 4,096$ solvable states on a 5×5 board out of 2^{25} possible grids. The solution adopted here constructs puzzles by starting from the solved state and applying ten random toggle operations in reverse. Since toggle is self-inverse, the puzzle is always solvable and has a solution of at most ten moves.

5. Conclusion

The Lights Out project demonstrates that a complete, interactive game can be implemented in bare-metal ARM Assembly without any library support. Every aspect of hardware interaction — LED control, button reading, timing — is performed through direct register access, making the boundary between software and hardware fully transparent.

The project concretely illustrates memory-mapped I/O (every peripheral is just an address), bit manipulation (the game state is stored as five packed row bytes and operates on exclusively with EOR, ORR, BIC, and LSL), hardware timer usage, and LED multiplexing as a form of time-division resource sharing. The LFSR demonstrates how a stateful mathematical function can be implemented in a handful of instructions with no division or modulo.

Overall, the exercise deepened our understanding of how high-level abstractions — arrays, loops, random numbers, timers — map to the actual instructions and memory transactions that a processor executes.